MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Temciuc Adelina, FAF-242

Report

on laboratory work no. 5
Operating Systems
Topic: The File System with Your Own Hands

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to check, on a real Linux system, the ideas about file systems from the lecture: the directory tree and the types of files, the inode that stands behind every name, hard and symbolic links, access permissions, and the disks and partitions that lie underneath. The work was done with ordinary commands only, by careful observation of their output.

The laboratory work aims to:

- explore the directory tree and recognise the type of a file from the first letter of `ls -l`;
- see the inode of a file with `stat` and `ls -i`, and compare hard links with symbolic links;
- understand the difference between the size of a file and the disk space it uses;
- experiment with permissions on files and directories, with the umask and with the special bits setuid and sticky;
- look at the disks, partitions and mounted file systems, and read information about the kernel from `/proc`.

THEORETICAL BACKGROUND

The directory tree and the types of files

Linux has a single directory tree that starts at `/`. Every disk or partition is attached (mounted) somewhere in this tree. "Everything is a file": besides regular files and directories, the tree contains symbolic links, character devices (such as terminals or `/dev/null`), block devices (disks), named pipes and sockets. The first character of a line printed by `ls -l` shows the type: `-` regular file, `d` directory, `l` symbolic link, `c` character device, `b` block device, `p` pipe and `s` socket.

Inodes, names and links

The data and the attributes of a file (type, owner, permissions, size, times, the list of data blocks and the number of links) are stored in an inode, identified by its number. The name of a file is not part of the inode: a directory is only a table of pairs "name – inode number". A hard link is one more such pair pointing to the same inode, so all hard links are equal and the file is deleted only when its link count drops to zero and no process keeps it open. A symbolic link is a separate small file whose content is a path; the kernel follows this path every time the link is used.

Permissions

Each file has an owner, a group and three groups of bits `rwx` for the owner (u), the group (g) and all others (o), usually written as three octal digits (for example 644). The kernel checks only one of the three groups: the owner group if the user is the owner, otherwise the group bits if the user is in the group, otherwise the others bits. On a directory, `r` allows reading the list of names, `w` allows creating and deleting entries and `x` allows passing through the directory to reach the files in it. The umask removes bits from the default permissions of new files (666) and directories (777). Three special bits exist: setuid, setgid and sticky.

Disks, mounting and /proc

A disk (for example `/dev/sda`) is divided into partitions (`/dev/sda1`, `/dev/sda2`), and a file system such as ext4 is created inside a partition and mounted into the tree. `lsblk` shows the disks and partitions, `df` shows how full the mounted file systems are, and `mount` lists them with their types and options. `/proc` is a virtual file system: its files are not stored on any disk, the kernel generates their content when they are read, which makes information about the CPU, memory and processes available as ordinary text files.

PREPARATION

The work was done in an Ubuntu virtual machine in VirtualBox, as the normal user `vboxuser`, not as root, because root ignores most permission checks. A separate working folder `~/os-lab5` was created for the experiments:

```
whoami             # must not print root
mkdir -p ~/os-lab5 && cd ~/os-lab5
```

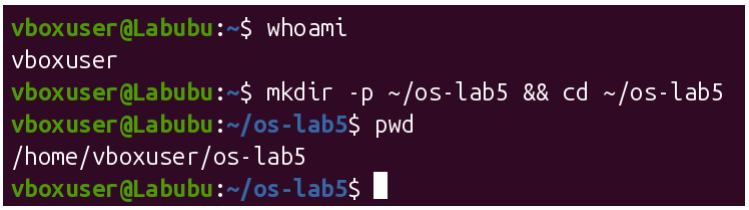


Figure 0.1 – Checking the user and creating the working folder

PART 1. THE TREE AND THE FILE TYPES

1.1 Commands executed

```
ls /
ls -ld /etc /etc/passwd /bin /dev/null /tmp
ls -l /dev | head
file /usr/bin/ls /etc/passwd /bin
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```

```
vboxuser@Labubu:~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
vboxuser@Labubu:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 12 10:35 /dev/null
drwxr-xr-x 141 root root 12288 Oct 12 07:36 /etc
-rw-r--r--   1 root root  2823 Oct  5 13:57 /etc/passwd
drwxrwxrwt  17 root root   400 Oct 12 12:00 /tmp
vboxuser@Labubu:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root     root     10, 235 Oct 12 10:35 autofs
drwxr-xr-x  2 root     root        500 Oct 12 11:59 block
drwxr-xr-x  2 root     root         80 Oct 12 10:35 bsg
crw-------  1 root     root     10, 234 Oct 12 10:35 btrfs-control
drwxr-xr-x  3 root     root         60 Oct 12 10:35 bus
lrwxrwxrwx+ 1 root     root          3 Oct 12 10:35 cdrom -> sr0
drwxr-xr-x  2 root     root       3780 Oct 12 11:59 char
crw-------  1 root     tty       5,   1 Oct 12 10:36 console
lrwxrwxrwx  1 root     root         11 Oct 12 10:35 core -> /proc/kcore
vboxuser@Labubu:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
vboxuser@Labubu:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
119M    /var/log
4.8G    /usr
```

Figure 1.1 – The root directory, file types and the size of /var/log and /usr

The first ten lines of `ls -l /dev` contain only character devices, directories and links, because block devices such as `sda` come later in alphabetical order. To find a block device, the disk and its partitions were listed separately:

```
ls -l /dev/sda /dev/sda1 /dev/sda2
```

```
vboxuser@Labubu:~/os-lab5$ ls -l /dev/sda /dev/sda1 /dev/sda2
brw-rw---- 1 root disk 8, 0 Oct 12 10:35 /dev/sda
brw-rw---- 1 root disk 8, 1 Oct 12 10:35 /dev/sda1
brw-rw---- 1 root disk 8, 2 Oct 12 10:35 /dev/sda2
vboxuser@Labubu:~/os-lab5$
```

Figure 1.2 – The block devices of the disk

1.2 Answers to the questions

1. The type of each path from the first letter of ls -l

Path	First letter	Type
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file (ASCII text)
`/bin`	`l`	symbolic link (to `usr/bin`)
`/dev/null`	`c`	character device, major 1, minor 3
`/tmp`	`d`	directory (with the sticky bit, `drwxrwxrwt`)

2. What is /bin on this system?

`/bin` is not a directory but a symbolic link: `/bin -> usr/bin`. Its size is 7 bytes, exactly the length of the text `usr/bin`. Modern Ubuntu uses the "merged /usr" layout, where `/bin`, `/sbin` and `/lib` are only links to the directories in `/usr`. The command `file` confirms it: `/bin: symbolic link to usr/bin`. It also showed that even `/usr/bin/ls` is a link, to `../lib/cargo/bin/coreutils/ls`, because this Ubuntu release uses the Rust version of the core utilities.

3. One character device and one block device in /dev

Character devices: `/dev/null` (`crw-rw-rw-`, numbers 1, 3), `/dev/console` (5, 1) and `/dev/autofs` (10, 235), all with the letter `c`. Block device: `/dev/sda` (`brw-rw----`, numbers 8, 0), the virtual hard disk, together with its partitions `/dev/sda1` (8, 1) and `/dev/sda2` (8, 2), with the letter `b`. Instead of a size, `ls -l` prints the major and minor numbers of a device: the major number selects the driver in the kernel, the minor number selects the device inside that driver.

4. Which is bigger, /var/log or /usr?

`/var/log` takes 119 MB and `/usr` takes 4.8 GB, so `/usr` is about 40 times bigger (about 4.7 GB more). `/usr` contains all installed programs and libraries, while `/var/log` contains only log files.

PART 2. INODES AND LINKS

2.1 Commands executed

Steps 1–3: an inode and its names, a hard link and a symbolic link.

```
echo hello > notes.txt
stat notes.txt
ls -li notes.txt
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt ; ls -l soft.txt
```

```
vboxuser@Labubu:~/os-lab5$ echo hello > notes.txt
vboxuser@Labubu:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 3148266     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/vboxuser)   Gid: ( 1000/vboxuser)
Access: 2026-10-12 12:10:25.415241597 +0000
Modify: 2026-10-12 12:10:25.338239065 +0000
Change: 2026-10-12 12:10:25.338239065 +0000
 Birth: 2026-10-12 12:10:25.338239065 +0000
vboxuser@Labubu:~/os-lab5$ ls -li notes.txt
3148266 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 12:10 notes.txt
vboxuser@Labubu:~/os-lab5$ ln notes.txt hard.txt
vboxuser@Labubu:~/os-lab5$ ls -li notes.txt hard.txt
3148266 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 12 12:10 hard.txt
3148266 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 12 12:10 notes.txt
vboxuser@Labubu:~/os-lab5$ rm notes.txt
vboxuser@Labubu:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
3148266 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 12:10 hard.txt
vboxuser@Labubu:~/os-lab5$ ln -s hard.txt soft.txt
vboxuser@Labubu:~/os-lab5$ ls -li hard.txt soft.txt
3148266 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 12:10 hard.txt
3146282 lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 12 12:17 soft.txt -> hard.txt
vboxuser@Labubu:~/os-lab5$ cat soft.txt
hello
vboxuser@Labubu:~/os-lab5$ rm hard.txt
vboxuser@Labubu:~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 12 12:17 soft.txt -> hard.txt
vboxuser@Labubu:~/os-lab5$
```

Figure 2.1 – stat, hard link and symbolic link

Steps 4–5: the link count of a directory, and size versus space.

```
mkdir d ; ls -ld d
mkdir d/x d/y ; ls -ld d
```

```
echo x > one.txt ; ls -l one.txt ; du -h one.txt
truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
rm big.img
```

```
vboxuser@Labubu:~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 12 12:18 d
vboxuser@Labubu:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 vboxuser vboxuser 4096 Oct 12 12:21 d
vboxuser@Labubu:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 vboxuser vboxuser 2 Oct 12 12:21 one.txt
4.0K    one.txt
vboxuser@Labubu:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 vboxuser vboxuser 1.0G Oct 12 12:21 big.img
0       big.img
vboxuser@Labubu:~/os-lab5$ rm big.img
```

Figure 2.2 – Directory links and sparse files

2.2 Answers to the questions

1. Which number in stat is the inode, and which tells how many names the file has?

The inode number is `Inode: 3148266`; the same number is printed in the first column of `ls -li`. The number of names is `Links: 1`, the link count; in `ls -l` it is the number right after the permissions.

2. Why could hard.txt still be read after rm notes.txt? What does rm really remove?

After `ln notes.txt hard.txt` both names showed the same inode 3148266 and the link count became 2: they are two equal names of one file, not an original and a copy. `rm` does not delete a file, it calls `unlink()`, which removes only one name from the directory and decreases the link count. After `rm notes.txt` the count was 1, the inode and its data blocks stayed untouched, so `cat hard.txt` still printed `hello`. The data is freed only when the link count reaches 0 and no process has the file open.

3. Why did cat soft.txt fail although soft.txt still exists? How big is soft.txt?

`soft.txt` has its own inode (3146282) and type `l`. It contains only the path `hard.txt`, not the data. When it is opened the kernel follows that path; after `rm hard.txt` there is no file with that name, so `cat` reported `No such file or directory`, and `ls` showed the broken link in red. The size of `soft.txt` is 8 bytes, exactly the number of characters in the text `hard.txt` that it stores.

4. Where do the 2 links of a new directory and the 4 links after two subdirectories come from?

A new directory `d` has 2 links: its name `d` in the directory `os-lab5`, and the entry `.` inside `d` itself. Every subdirectory contains an entry `..` that points to its parent, so after `mkdir d/x d/y`

the entries `d/x/..` and `d/y/..` added two more links: 2 + 2 = 4. In general, the link count of a directory is 2 plus the number of its subdirectories (this is also why `/etc` showed 141).

5. Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

`ls -l` shows the size, the number of bytes in the file, while `du` shows the space actually allocated on disk. ext4 allocates space in whole blocks of 4096 bytes (`IO Block: 4096` in `stat`), so the 2 bytes `x\n` of `one.txt` occupy one full 4 KB block. `truncate -s 1G` only sets the size field of the inode and allocates no blocks: `big.img` is a sparse file whose "holes" are read as zeros, so its size is 1.0G but `du` shows 0.

PART 3. PERMISSIONS

3.1 Commands executed

```
echo secret > s.txt
chmod 600 s.txt ; ls -l s.txt
chmod 000 s.txt ; cat s.txt
chmod u+r s.txt ; cat s.txt
umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
mkdir box ; echo hi > box/f
chmod u=r,go= box ; ls box ; cat box/f      # read only
chmod u=x,go= box ; ls box ; cat box/f      # execute only
chmod 755 box
ls -l /usr/bin/passwd ; ls -ld /tmp
cat /etc/shadow
ls -l /etc/shadow
```

```
vboxuser@Labubu:~/os-lab5$ echo secret > s.txt
vboxuser@Labubu:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 vboxuser vboxuser 7 Oct 12 12:22 s.txt
vboxuser@Labubu:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
vboxuser@Labubu:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
vboxuser@Labubu:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 vboxuser vboxuser    0 Oct 12 12:23 new.txt
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 12 12:23 newdir
vboxuser@Labubu:~/os-lab5$ mkdir box ; echo hi > box/f
vboxuser@Labubu:~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f

cat: box/f: Permission denied
vboxuser@Labubu:~/os-lab5$
vboxuser@Labubu:~/os-lab5$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
vboxuser@Labubu:~/os-lab5$ chmod 755 box
vboxuser@Labubu:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 17 root root 400 Oct 12 12:10 /tmp
vboxuser@Labubu:~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
vboxuser@Labubu:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1336 Oct  5 13:57 /etc/shadow
vboxuser@Labubu:~/os-lab5$
```

Figure 3.1 – Permissions on files and directories, special bits

3.2 Answers to the questions

1. 600, 644, 755 and 700 as rwx strings, with a real use for each

Octal	rwx string	Real use
600	`rw-------`	private files that only the owner may read and change, e.g. `~/.ssh/id_ed25519`

Octal	rwx string	Real use
644	`rw-r--r--`	ordinary files that everybody may read, e.g. `/etc/passwd` or a web page
755	`rwxr-xr-x`	programs and public directories, e.g. `/usr/bin/passwd` (without s) or the folder `box`
700	`rwx------`	private directories or scripts, e.g. `~/.ssh`

2. With chmod 000 you own the file but cannot read it. Why does ownership not help?

The kernel chooses exactly one group of bits: since `vboxuser` is the owner, only the owner bits `---` are checked, and they give no right to read. Being the owner does not add any access by itself; it only gives the right to change the permissions with `chmod`. That is why `chmod u+r s.txt` was allowed and after it `cat` printed `secret`. Only root bypasses these checks.

3. The umask, and the permissions of new.txt and newdir

The umask is `0002`: it removes only the write bit for others. New files are created with 666 and new directories with 777, and the umask bits are removed: 666 − 002 = 664 (`-rw-rw-r--` for `new.txt`) and 777 − 002 = 775 (`drwxrwxr-x` for `newdir`). Ubuntu uses 002 because every user has a private group with the same name, so group write is safe.

4. The box experiment: what does r allow on a directory, and what does x allow?

With `u=r` (read only) `ls box` printed the name `f`, because reading a directory means reading its list of names, but `cat box/f` failed with `Permission denied`, because to reach a file through a directory the `x` bit is needed. With `u=x` (execute only) the opposite happened: `ls box` failed (`cannot open directory`), but `cat box/f` printed `hi`, since the file can be reached when its name is already known. So `r` on a directory allows listing the names, `x` allows passing through it (search) to the inodes of its files, and `w` together with `x` allows creating, renaming and deleting entries.

5. The s in /usr/bin/passwd and the t in /tmp

The `s` in place of the owner `x` (`-rwsr-xr-x root`) is the setuid bit: the program runs with the effective user ID of the owner of the file, root, no matter who started it. `passwd` needs it because the password hashes are stored in `/etc/shadow`, which belongs to root and is `rw-r-----`; as the experiment showed, an ordinary user cannot even read it. Thanks to setuid, `passwd` can update this file, but only in the controlled way the program allows.

The `t` in place of the others `x` on `/tmp` (`drwxrwxrwt`) is the sticky bit. Everybody may create files in `/tmp`, but in a sticky directory a file can be deleted or renamed only by its owner, the owner of the directory or root, so users cannot delete each other's temporary files.

PART 4. DISKS AND THE KERNEL AS FILES

4.1 Commands executed

```
lsblk
df -h
mount | grep " / "
grep -c processor /proc/cpuinfo
head -3 /proc/meminfo
head -5 /proc/self/status
ls -l /proc/self/fd
```

```
vboxuser@Labubu:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/1643
loop2    7:2    0    20M  1 loop /snap/desktop-security-center/151
loop3    7:3    0  66.8M  1 loop /snap/core24/2124
loop4    7:4    0  14.2M  1 loop /snap/desktop-security-center/206
loop5    7:5    0  16.5M  1 loop /snap/firmware-updater/226
loop6    7:6    0 260.3M  1 loop /snap/firefox/8763
loop7    7:7    0 614.5M  1 loop /snap/gnome-46-2404/164
loop8    7:8    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop9    7:9    0   1.5M  1 loop /snap/hwctl/123
loop10   7:10   0   1.5M  1 loop /snap/hwctl/131
loop11   7:11   0   402M  1 loop /snap/mesa-2404/1839
loop12   7:12   0  18.4M  1 loop /snap/prompting-client/228
loop13   7:13   0  18.8M  1 loop /snap/prompting-client/222
loop14   7:14   0  11.8M  1 loop /snap/snap-store/1390
loop15   7:15   0  50.1M  1 loop /snap/snapd/27710
loop16   7:16   0  44.7M  1 loop /snap/snapd/28254
loop17   7:17   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0  50.1G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0  50.1G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/vboxuser/VBox_GAs_7.2.20
vboxuser@Labubu:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.7M  677M   1% /run
/dev/sda2        49G  7.5G   40G  17% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
tmpfs           1.7G  8.0K  1.7G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sr0         51M   51M     0 100% /run/media/vboxuser/VBox_GAs_7.2.20
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           340M   96K  340M   1% /run/user/1000
```

Figure 4.1 – Disks, partitions and mounted file systems

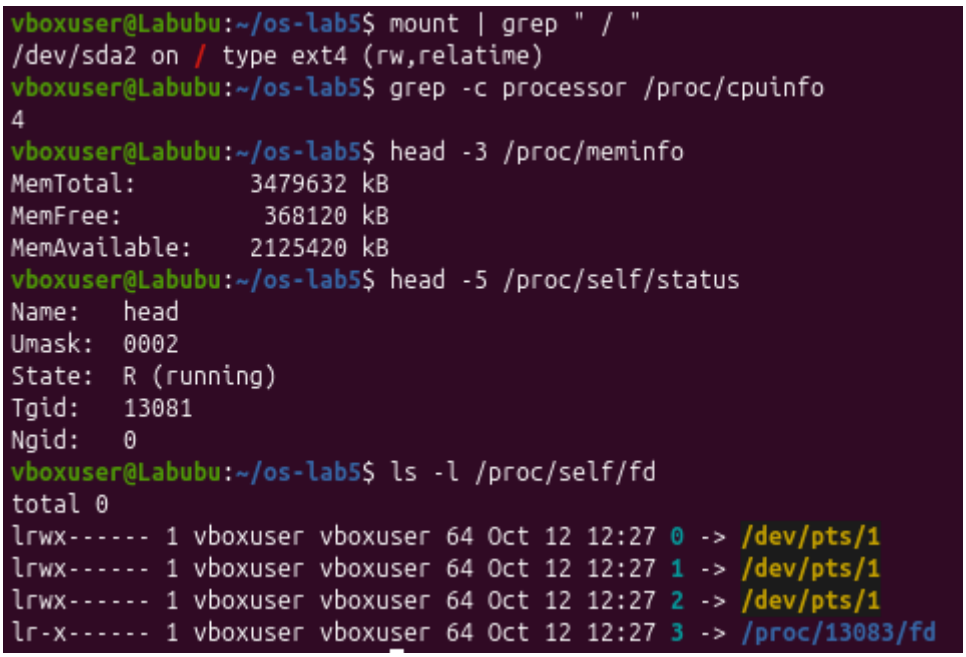


Figure 4.2 – The root mount, CPU, memory and /proc/self

4.2 Answers to the questions

1. The disk: name, partitions, the root partition and its file system

The virtual disk of the machine is `sda`, 50.1 GB. It has two partitions: `sda1` of 1 MB, the BIOS boot partition used by GRUB on a GPT disk, and `sda2` of 50.1 GB, which is mounted at `/` with the file system ext4 (`/dev/sda2 on / type ext4 (rw,relatime)`). The other devices are not real disks: `loop0`–`loop17` are the read-only images of the installed snap packages, and `sr0` is the virtual CD with the VirtualBox Guest Additions.

```
sda    50.1G  disk  (VirtualBox virtual disk, /dev/sda)
├─sda1   1M  part  BIOS boot partition (GRUB), not mounted
└─sda2 50.1G part  ext4, mounted at /   (rw,relatime)
sr0    50.7M  rom   VBox Guest Additions CD,
/run/media/vboxuser/VBox_GAs_7.2.20
loop0..loop17      snap packages, mounted under /snap
```

2. How full is the root file system?

`df -h` shows `/dev/sda2` with a size of 49G, 7.5G used and 40G available, so the root file system is 17% full. The other lines with `tmpfs` are file systems that live in RAM.

3. How many CPU cores and how much RAM, according to /proc?

`grep -c processor /proc/cpuinfo` printed 4, so the virtual machine has 4 CPU cores. `/proc/meminfo` shows `MemTotal: 3479632 kB`, about 3.3 GB of RAM (the machine was given 3.5 GB; the kernel reserves a part of it for itself). At that moment 2.1 GB were available.

4. Which program name did /proc/self/status show, and why?

It showed `Name: head`. `/proc/self` is a magic link that points to the directory of the process that is accessing it. The file was opened by the `head` process, not by the shell, so it described `head` (PID 13081). In the same way, `ls -l /proc/self/fd` showed the open files of `ls` itself: 0, 1 and 2 are the terminal `/dev/pts/1`, and 3 is the directory `/proc/13083/fd` that `ls` opened in order to list it.

PART 5. STRETCH TASKS

5.1 (a) Magic numbers

```
gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd notes.txt | head
-1
cat /etc/passwd | xxd | head -1
```

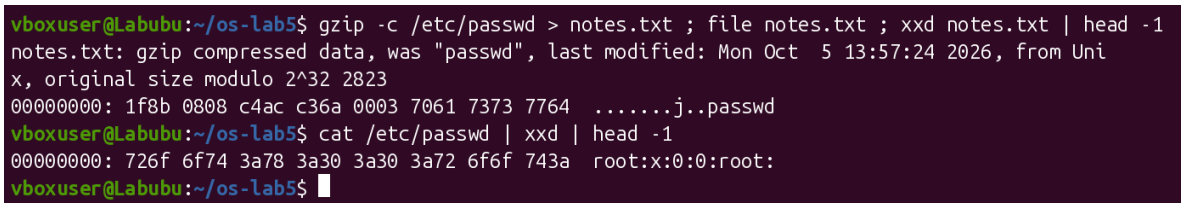


Figure 5.1 – file recognises gzip data by its first bytes

Although the file is called `notes.txt`, `file` reported `gzip compressed data, was "passwd"`. The type is recognised from the content, not from the name: the first two bytes `1f 8b` are the gzip magic number, the third byte `08` means the deflate method, and the flag `08` says that the original file name follows the header, which is why `passwd` is visible in the first line of `xxd`. A plain text file such as `/etc/passwd` has no magic number; `file` recognises it as text because all bytes are printable characters.

5.2 (c) Deleting a file that is still open

```
dd if=/dev/zero of=bigfile bs=1M count=100 status=none ; df -h . | tail
-1
exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd
df -h . | tail -1
exec 3<&- ; df -h . | tail -1
```

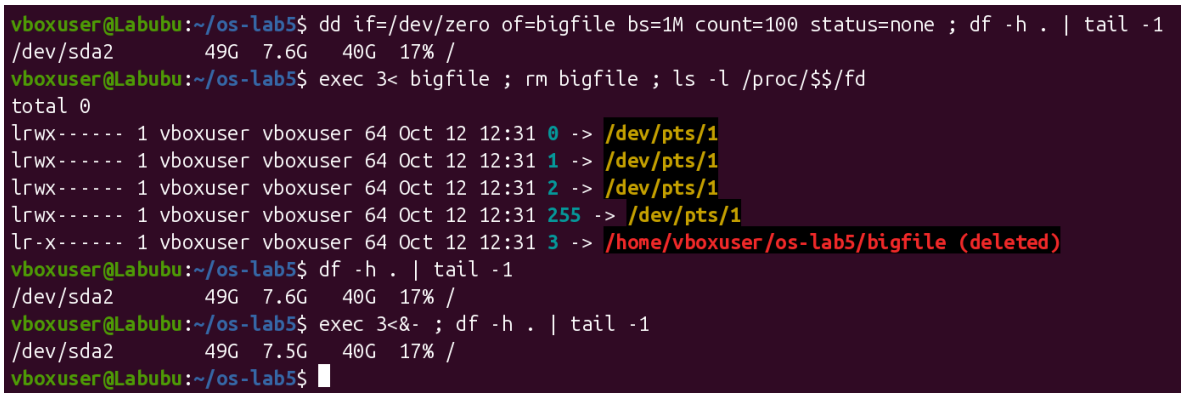


Figure 5.2 – An open file survives rm until it is closed

After `rm bigfile` the name was gone, but the shell still had the file open on descriptor 3, and the link in `/proc/$$/fd` showed `/home/vboxuser/os-lab5/bigfile (deleted)`. The 100 MB were still in use (7.6G used). Only after `exec 3<&-` closed the descriptor did the used space

go back to 7.5G. The space is really freed only when both the link count is 0 and the last open reference to the inode is closed.

CONCLUSION

In this laboratory work the Linux file system was studied with ordinary commands on an Ubuntu virtual machine.

In the first part, the tree and the types of files were explored. The first letter of `ls -l` showed directories, regular files, links and devices; `/bin` turned out to be only a link to `usr/bin`, and devices like `/dev/null` and `/dev/sda` showed the major and minor numbers of their drivers instead of a size.

In the second part, `stat` and `ls -i` showed that the name of a file is separate from its inode. A hard link is just a second name of the same inode, so deleting the "original" did not lose the data, while a symbolic link stores only a path (8 bytes for `hard.txt`) and breaks when its target is removed. The link count of a directory comes from its name, its `.` and the `..` of each subdirectory. The difference between size and space was shown by a 2-byte file that uses a 4 KB block and a 1 GB sparse file that uses nothing.

In the third part, the permission experiments showed that the kernel checks only one group of bits, that the umask 002 gives 664 for files and 775 for directories, that `r` and `x` mean listing and passing through for a directory, and why `passwd` needs the setuid bit and `/tmp` the sticky bit.

In the fourth part, `lsblk`, `df` and `mount` showed a 50 GB disk with an ext4 root partition that is 17% full, and the virtual files in `/proc` reported 4 CPU cores and about 3.3 GB of RAM. `/proc/self` showed that the kernel produces these files for whichever process reads them. The stretch tasks showed that the type of a file is identified by its magic number and that an open file keeps its space until it is closed.

REFERENCES

1. Operating Systems course, FCIM/FAF, UTM, 2026–2027. Lab 5: The File System with Your Own Hands.
2. stat(1), ln(1), chmod(1) – Linux manual pages. [online]. Available: https://man7.org/linux/man-pages/
3. proc(5) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html
4. path_resolution(7) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man7/path_resolution.7.html
5. Arpaci-Dusseau R., Arpaci-Dusseau A. Operating Systems: Three Easy Pieces, chapter 39 "Files and Directories". [online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/